



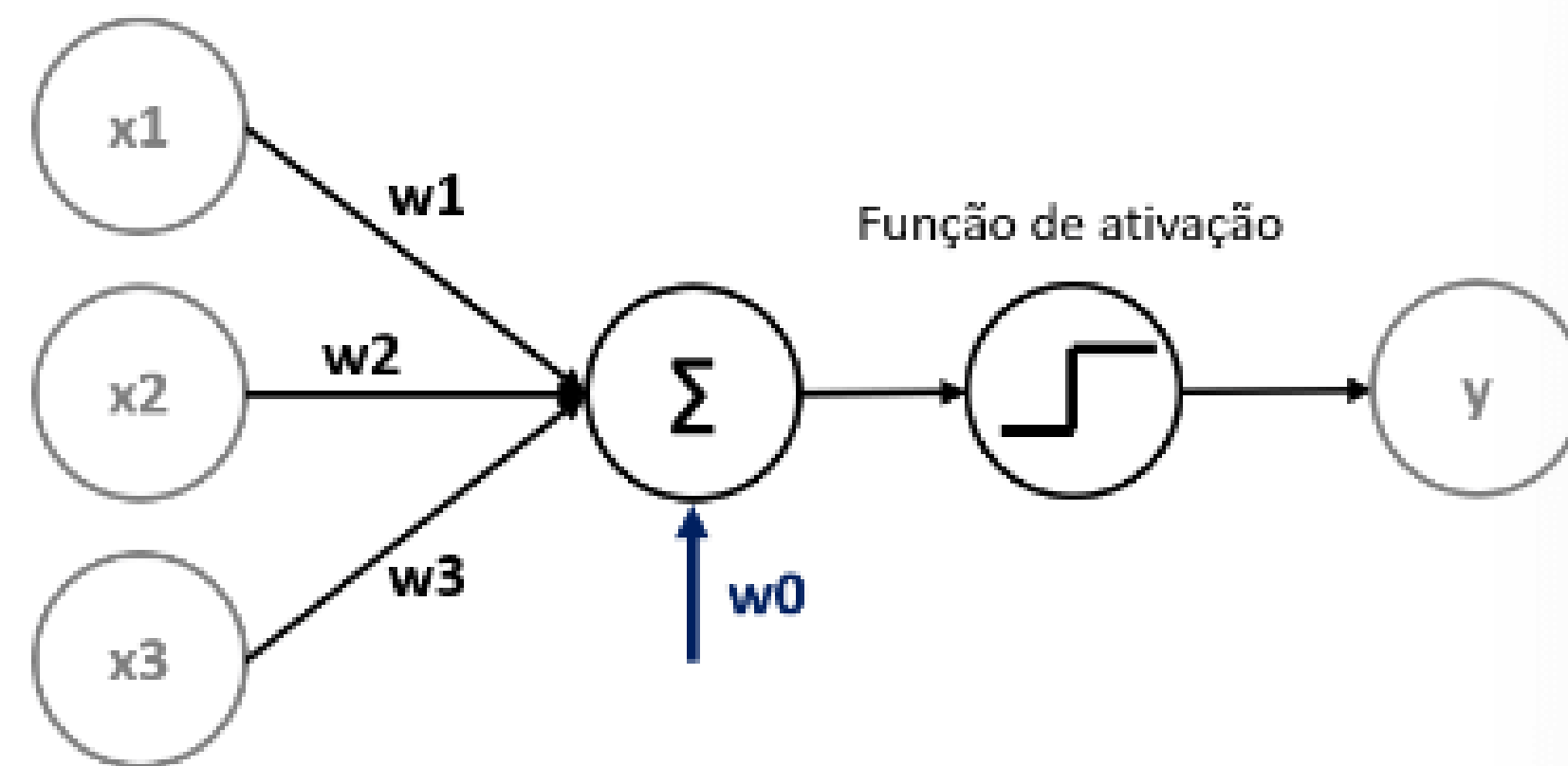
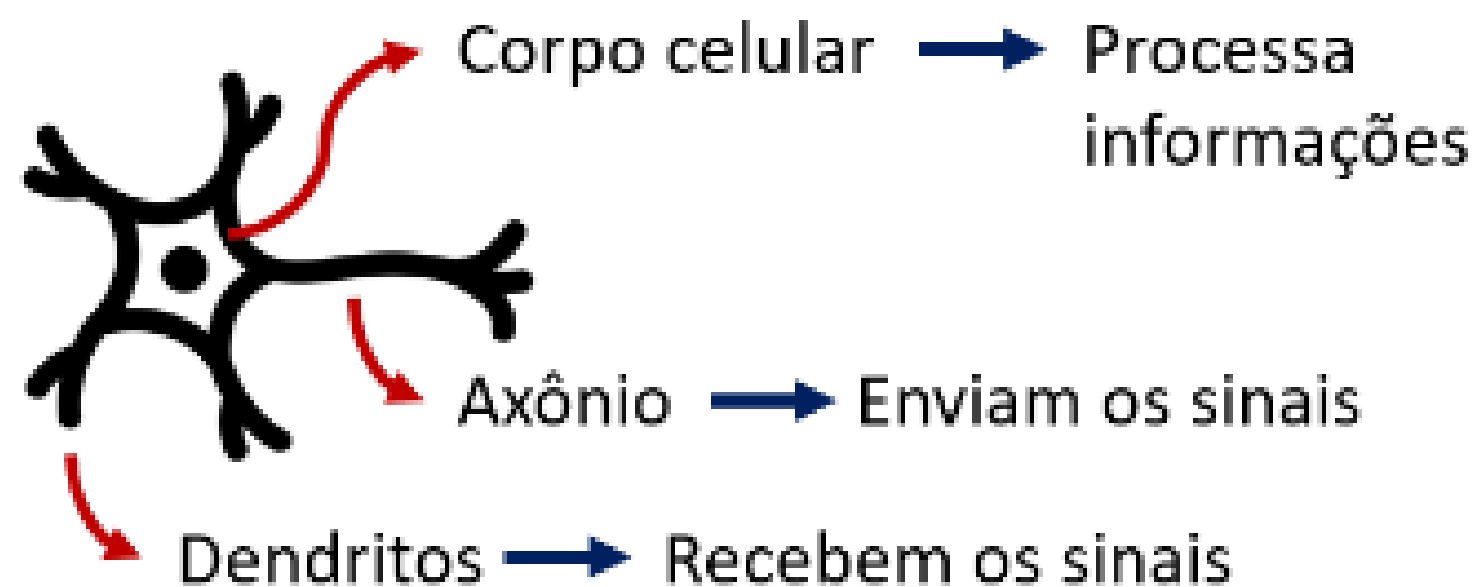
UNINASSAU

PERCEPTRON EM MÚLTIPLAS CAMADAS

Análise e Desenvolvimento de Sistemas
Machine Learning
Profº Lindenberg Andrade

O que é o Perceptron?

Um perceptron é um algoritmo de aprendizado supervisionado para classificação binária, considerado o modelo mais simples de uma rede neural artificial. Criado por Frank Rosenblatt em 1957, ele funciona como um neurônio artificial que aprende a categorizar dados de entrada em duas classes diferentes (por exemplo, "gato" ou "cachorro") ao encontrar uma "linha de decisão" que separe os dados de forma ideal. É a base para redes neurais mais complexas e pode ser utilizado para tarefas de aprendizado de máquina.



Por Que Precisamos de Múltiplas Camadas?

Limitação do Perceptron Simples

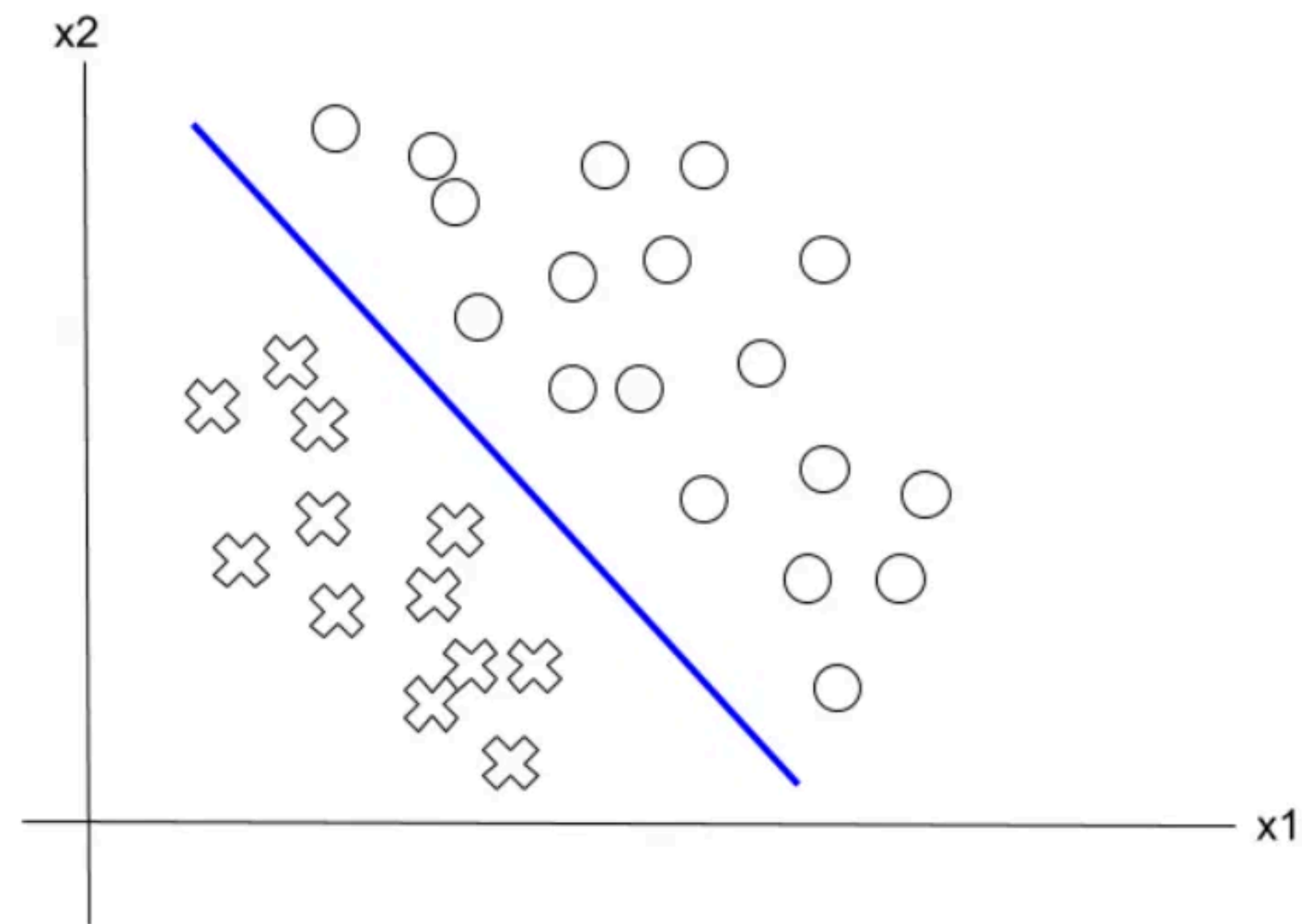
Um Perceptron com uma única camada consegue resolver apenas problemas linearmente separáveis. Exemplos: portas lógicas AND e OR.

O Desafio: Problema XOR

A função OU Exclusivo (XOR) não é linearmente separável. Um único Perceptron falha completamente em classificá-la corretamente, não importa como ajustemos os pesos.

A Solução: Múltiplas Camadas

A adição de camadas ocultas permite que a rede aprenda representações não-lineares dos dados, resolvendo o XOR e problemas muito mais complexos.



Separabilidade Linear: Dois conjuntos de pontos são linearmente separáveis se existe um hiperplano que os separa perfeitamente.

Arquitetura do MLP: Estrutura em Camadas

Entrada

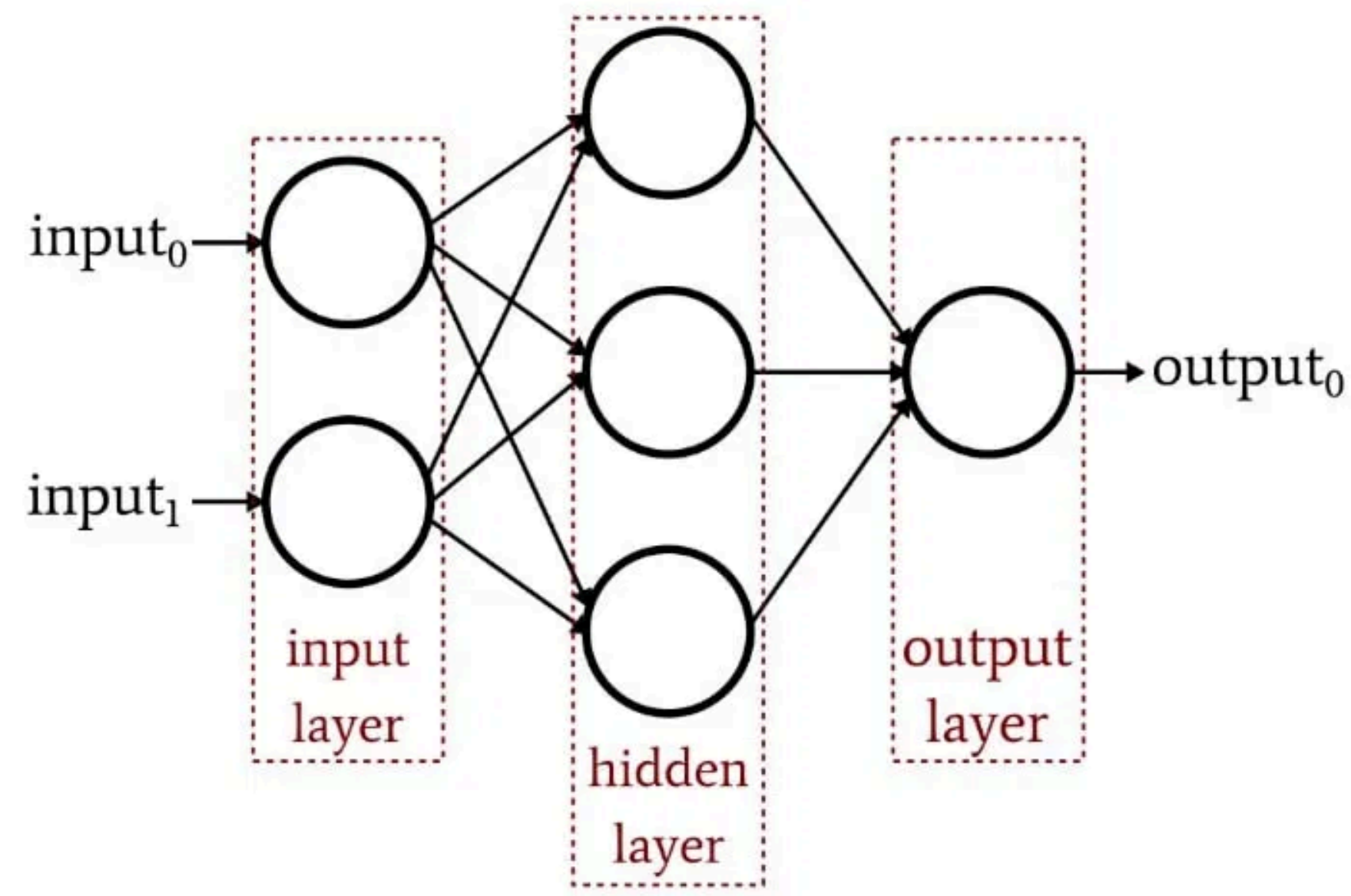
Recebe os dados brutos (features) do problema. Não realiza processamento, apenas distribui os valores para a próxima camada.

Ocultas

Onde o "pensamento" da rede acontece. Cada camada aprende representações progressivamente mais complexas dos dados.

Saída

Produz o resultado final da rede (ex: classificação, regressão, previsão).



Conexões Fully Connected (Densa)

Cada neurônio de uma camada está conectado a todos os neurônios da camada seguinte. Isso permite que cada neurônio receba informações de todos os neurônios anteriores.

Anatomia do Neurônio Artificial

Entradas (Inputs)

Recebe sinais (valores) dos neurônios da camada anterior ou dos dados de entrada. Cada entrada representa uma feature do problema.

Pesos (Weights - w)

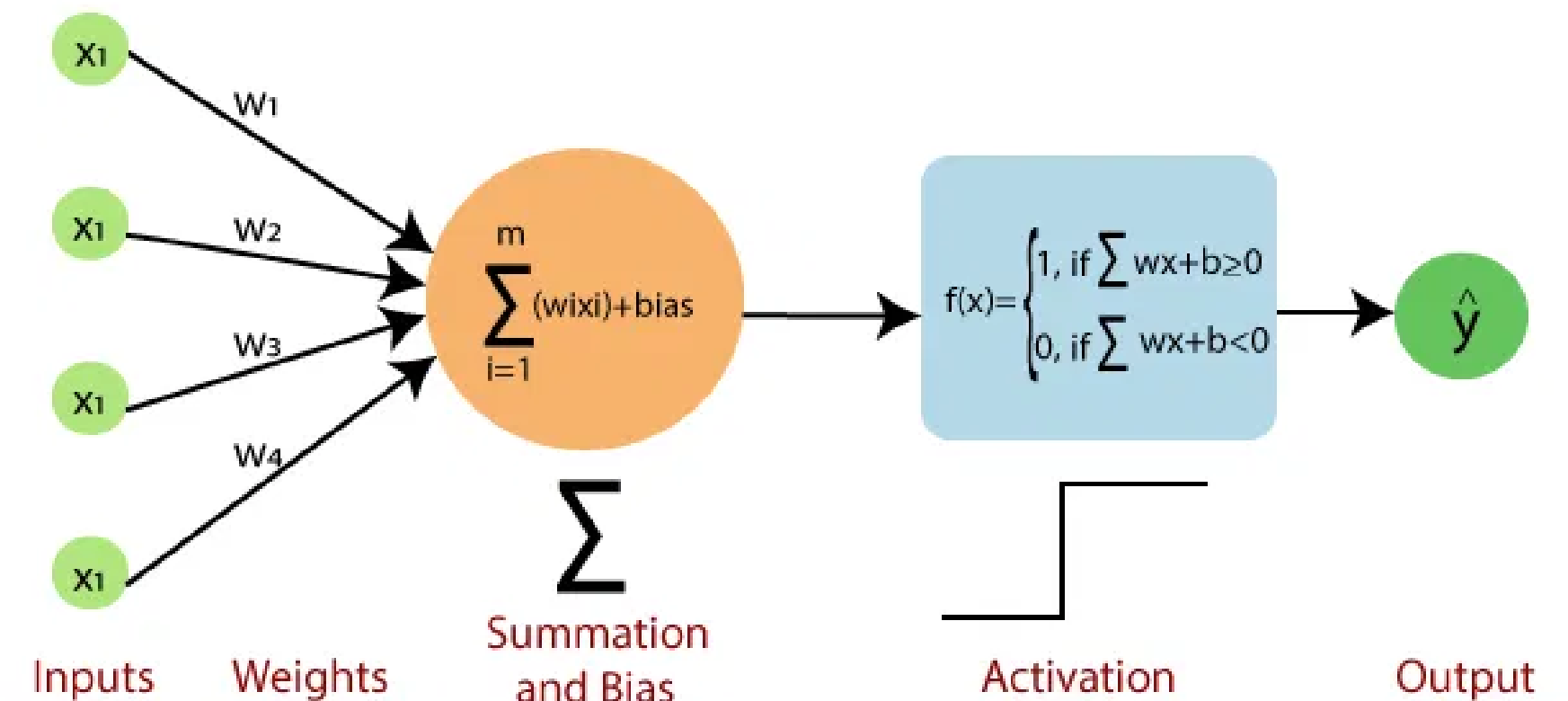
Cada conexão de entrada tem um peso associado que determina a importância daquela entrada. Pesos maiores amplificam o sinal; menores o atenuam.

Viés (Bias - b)

Um parâmetro adicional que permite ao neurônio deslocar sua função de ativação. Sem o viés, a rede seria limitada em suas representações.

Função de Ativação (σ)

Função não-linear que decide se o neurônio "dispara". Exemplos: Sigmoid, ReLU, Tanh. Essencial para criar representações complexas.



$$z = (\sum w_i \cdot x_i) + b$$
$$a = \sigma(z)$$

Propagação Direta: Do Input ao Output

Fórmulas Matemáticas

Soma Ponderada (Net Input)

$$z = (\sum w_i \times x_i) + b$$

z = soma ponderada

w_i = peso da i -ésima entrada

x_i = i -ésima entrada

b = viés (bias)

Saída do Neurônio (Ativação)

$$a = \sigma(z)$$

a = saída do neurônio

σ = função de ativação

Processo Iterativo

Este processo é repetido camada por camada, movendo a informação da entrada para a saída. A saída de uma camada torna-se a entrada da próxima.

Ponto-Chave

A **forward propagation** é determinística: dado um input e um conjunto de pesos, sempre produzirá o mesmo output. É durante o treinamento (backpropagation) que os pesos são ajustados para melhorar as previsões.

Propagação Direta: Do Input ao Output

Conceito Prático

Exemplo Passo a Passo

1. Entrada: $x_1=0.5$, $x_2=0.3$
2. Pesos: $w_1=0.8$, $w_2=0.6$
3. Viés: $b=0.1$
4. Soma: $z = (0.8 \times 0.5) + (0.6 \times 0.3) + 0.1 = 0.68$
5. Ativação: $a = \sigma(0.68) \approx 0.664$ (Sigmoid)

Fluxo de Informação

Entrada $\rightarrow$ Pesos $\times$ Entrada $\rightarrow$ Soma + Viés $\rightarrow$ Ativação $\rightarrow$ Saída

A propagação direta calcula a saída da rede para um dado input, passando por todas as camadas sequencialmente.

Propagação Direta: Do Input ao Output

Conceito Prático

Exemplo Passo a Passo

1. Entrada: $x_1=0.5$, $x_2=0.3$
2. Pesos: $w_1=0.8$, $w_2=0.6$
3. Viés: $b=0.1$
4. Soma: $z = (0.8 \times 0.5) + (0.6 \times 0.3) + 0.1 = 0.68$
5. Ativação: $a = \sigma(0.68) \approx 0.664$ (Sigmoid)

Fluxo de Informação

Entrada $\rightarrow$ Pesos $\times$ Entrada $\rightarrow$ Soma + Viés $\rightarrow$ Ativação $\rightarrow$ Saída

A propagação direta calcula a saída da rede para um dado input, passando por todas as camadas sequencialmente.

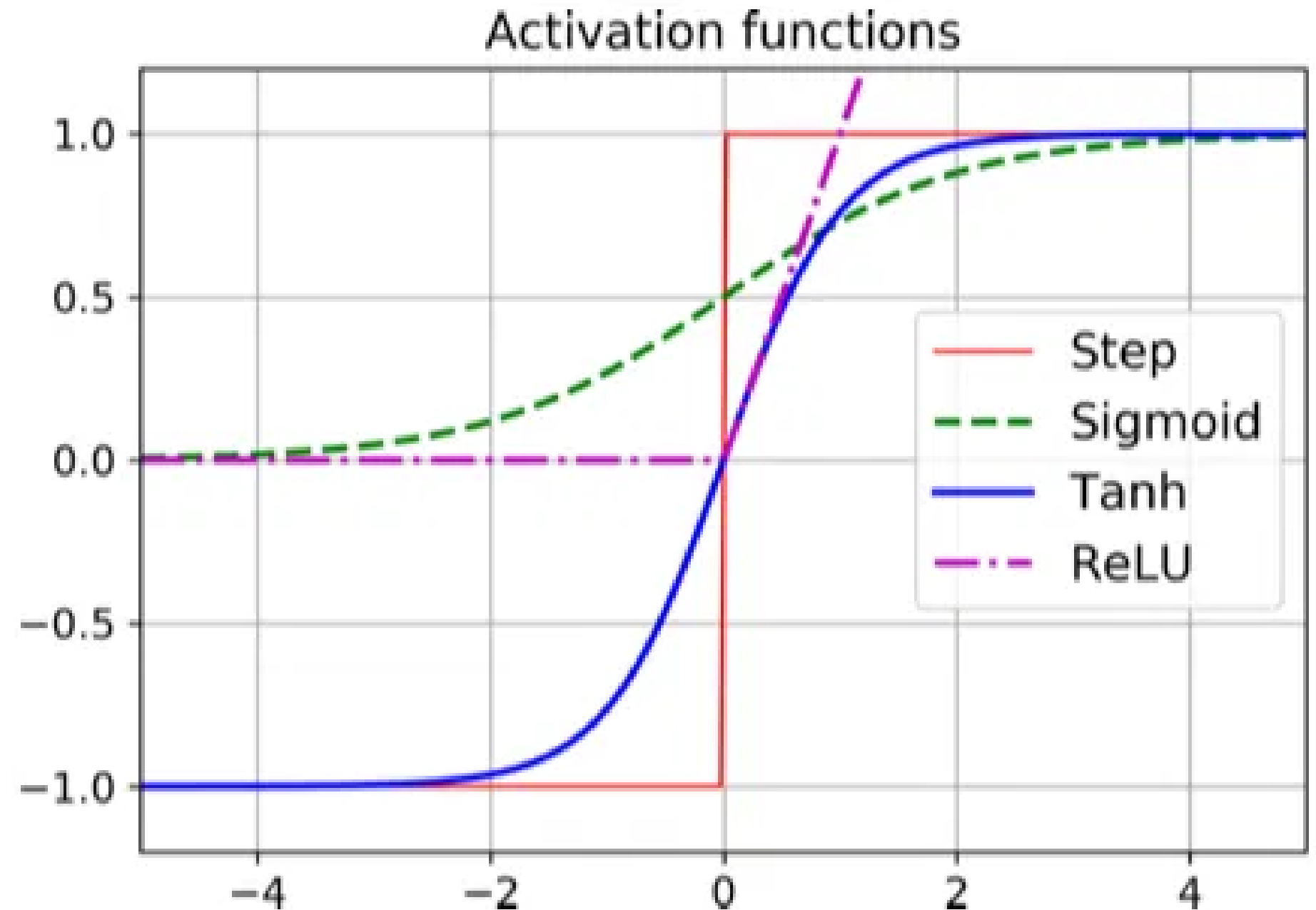
Funções de Ativação: Introduzindo a Não-Linearidade

Propósito: Introduzem a não-linearidade. Sem elas, o MLP seria apenas uma série de transformações lineares, equivalente a um Perceptron Simples.

Sigmoide

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

Comprime o valor de saída entre **0 e 1**.
Útil para a camada de saída em problemas de classificação binária.
Historicamente importante, mas menos usada em camadas ocultas.



Funções de Ativação: Introduzindo a Não-Linearidade

ReLU (Rectified Linear Unit)

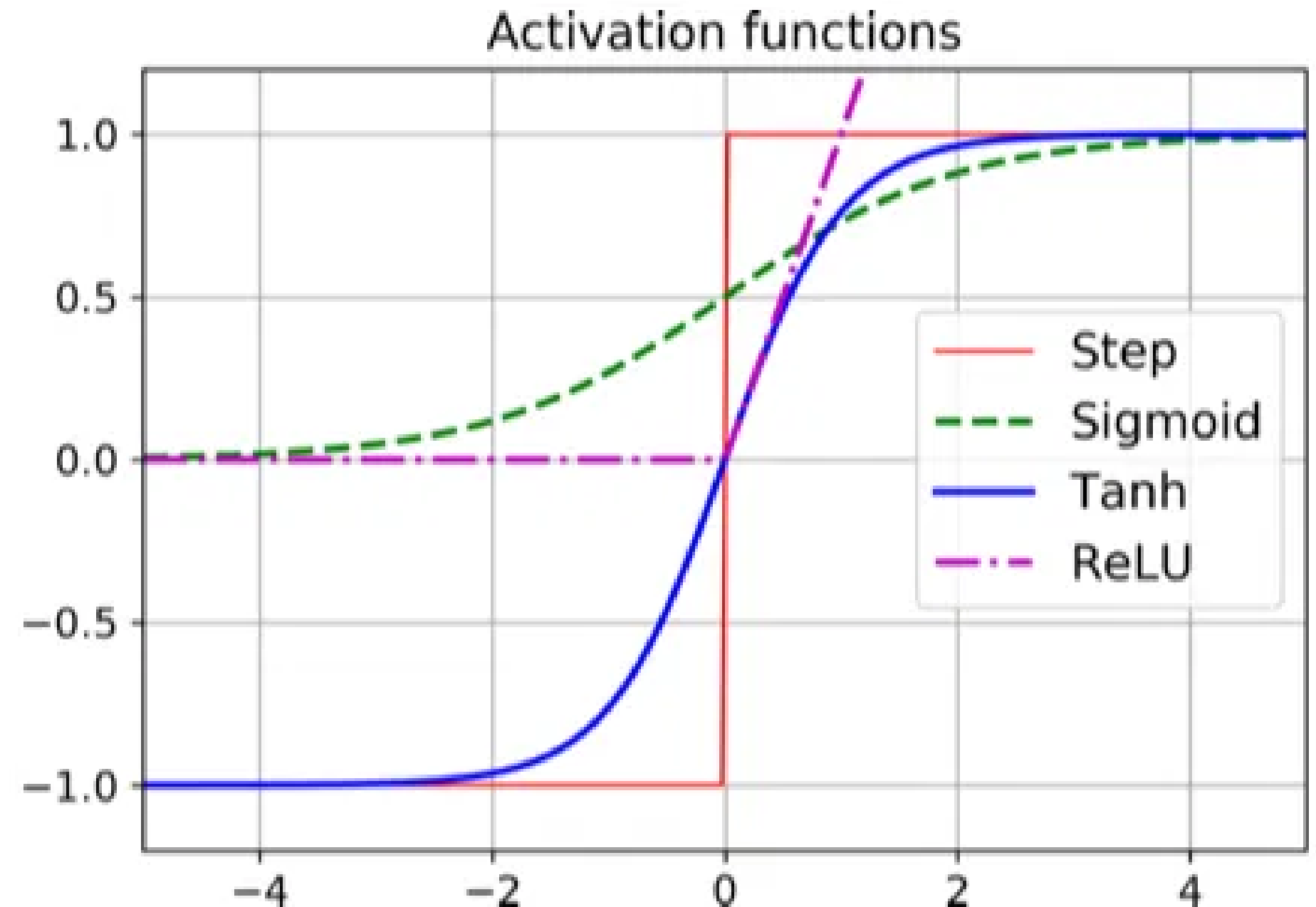
$$f(x) = \max(0, x)$$

A função mais popular em camadas ocultas. Extremamente **eficiente computacionalmente** e ajuda a mitigar o problema do **vanishing gradient** durante o treinamento.

Tanh (Tangente Hiperbólica)

$$\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$$

Comprime entre **-1 e 1**. Oferece melhor desempenho que Sigmoid em muitos casos, pois é centrada em zero, facilitando o aprendizado.



Treinamento do MLP: Minimizando o Erro

Objetivo do Treinamento

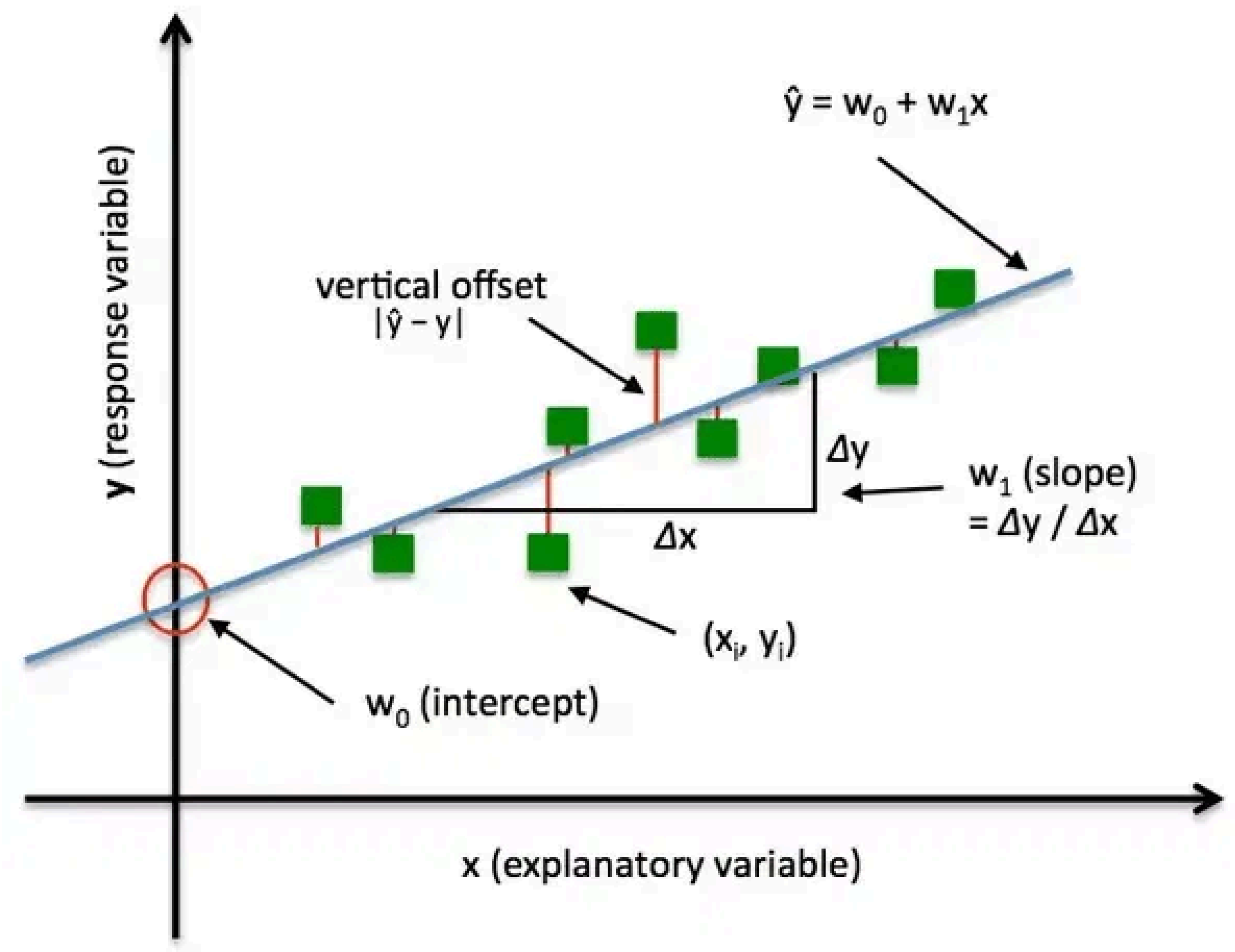
Ajustar os **pesos (w)** e os **vieses (b)** da rede para que a saída prevista ($\hat{y}$) seja o mais próxima possível da saída real (y).

Função de Custo/Perda

Mede quantitativamente o quão "**errada**" está a previsão da rede. Quanto menor o valor da função de custo, melhor o desempenho da rede.

Exemplo: Erro Quadrático Médio (MSE)

$$L = (1/n) \sum (y - \hat{y})^2$$



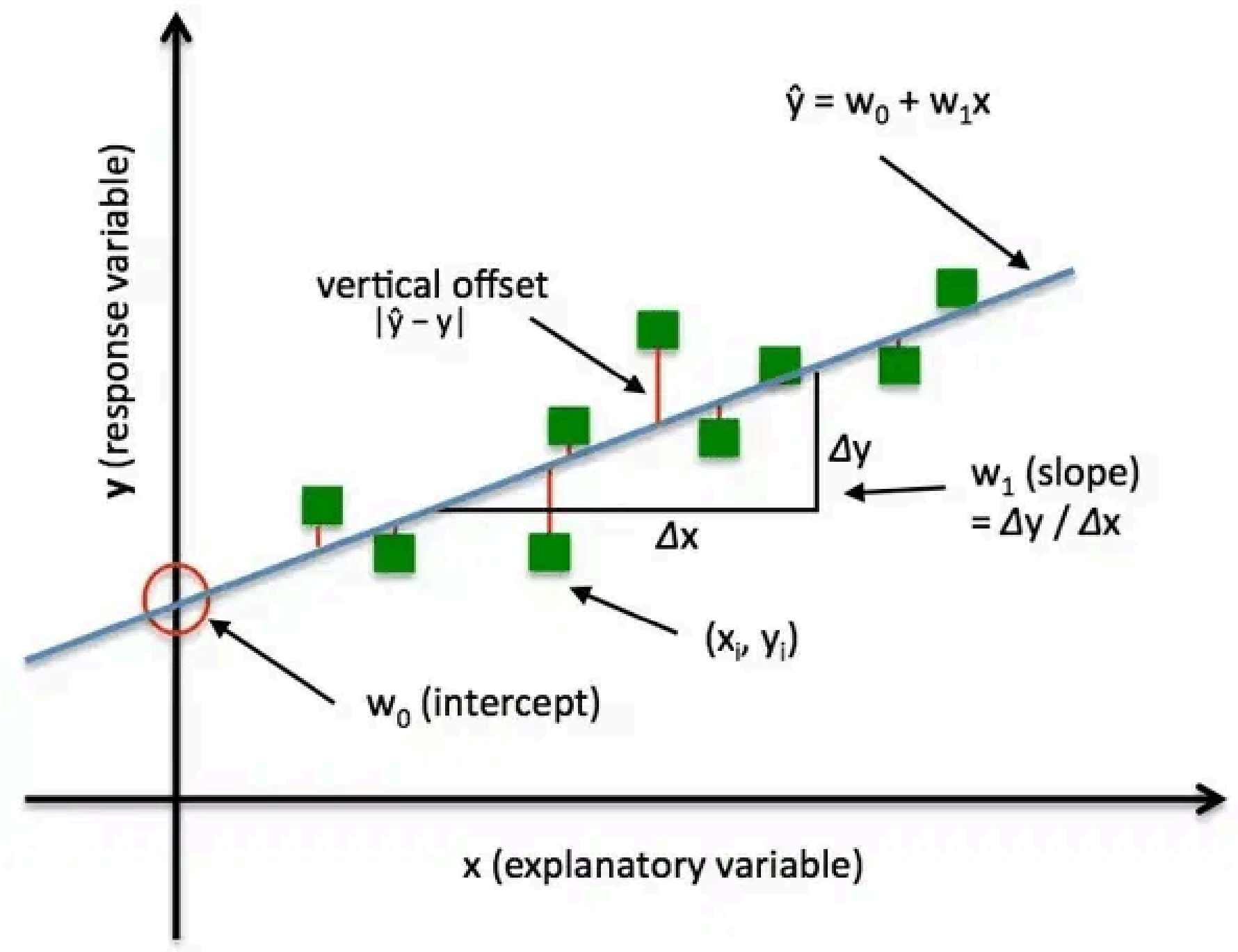
Treinamento do MLP: Minimizando o Erro

O Desafio

Encontrar o conjunto de pesos que **minimiza** a função de custo. Este é um problema de otimização complexo em um espaço multidimensional.

Por Que é Difícil?

Uma rede com milhões de parâmetros pode ter bilhões de possíveis combinações de pesos. Precisamos de um algoritmo eficiente para encontrar a melhor solução.



Backpropagation: O Algoritmo de Ajuste de Pesos

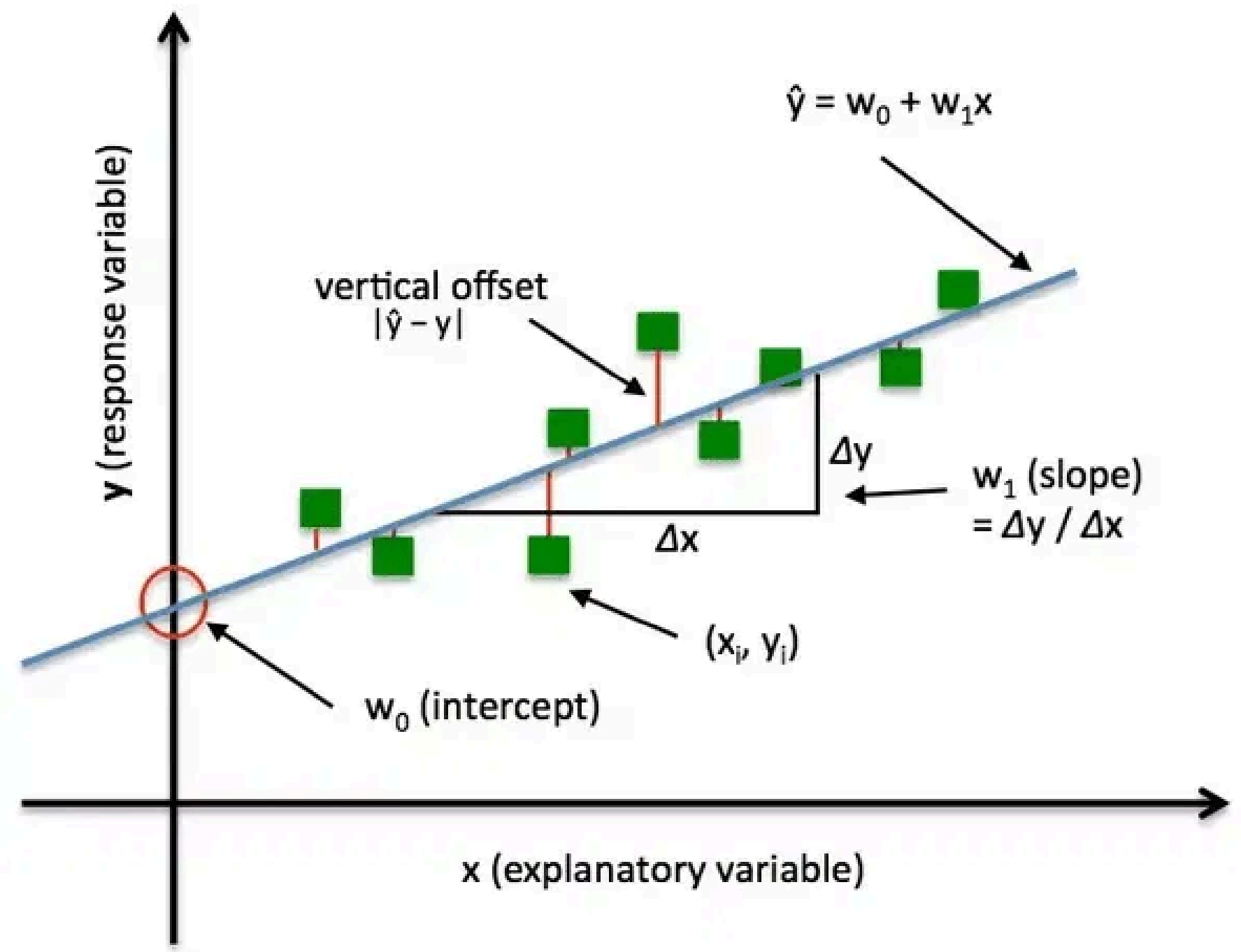
O Conceito

É o método utilizado para calcular a contribuição de **cada peso** no erro total da rede. O erro é propagado **de volta** (backward) da camada de saída para as camadas ocultas, camada por camada.

Cálculo do Gradiente

Utiliza a **Regra da Cadeia** do cálculo para determinar o gradiente (a derivada da função de custo em relação a cada peso).

$$\partial L / \partial w = (\partial L / \partial a) \cdot (\partial a / \partial z) \cdot (\partial z / \partial w)$$

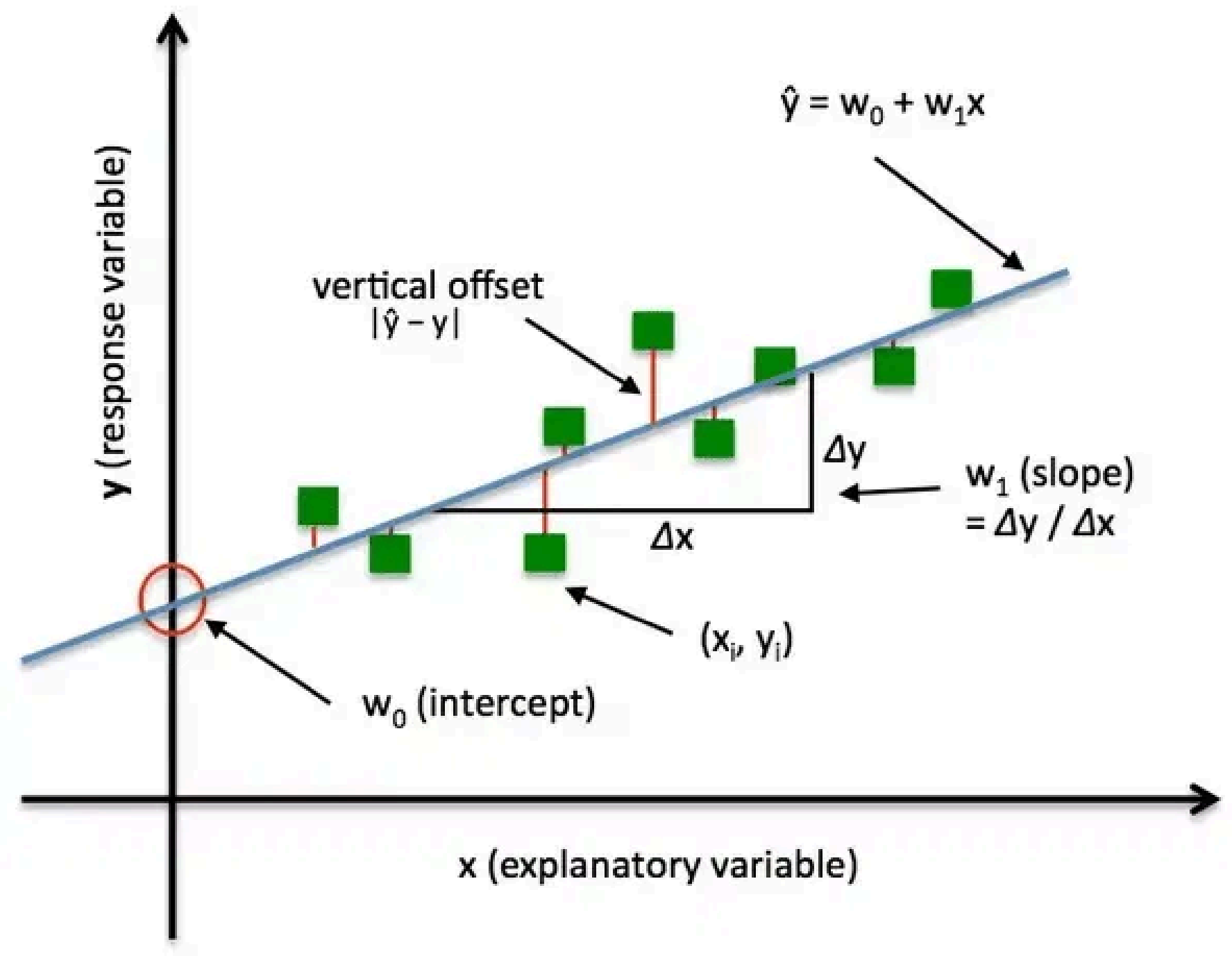


Backpropagation: O Algoritmo de Ajuste de Pesos

O Gradiente: Direção e Intensidade

O gradiente indica **para onde** e **quanto** o peso deve ser ajustado para reduzir o erro. É o "mapa" que guia a otimização da rede.

Insight: Backpropagation é eficiente porque reutiliza cálculos intermediários, tornando o treinamento de redes profundas computacionalmente viável.



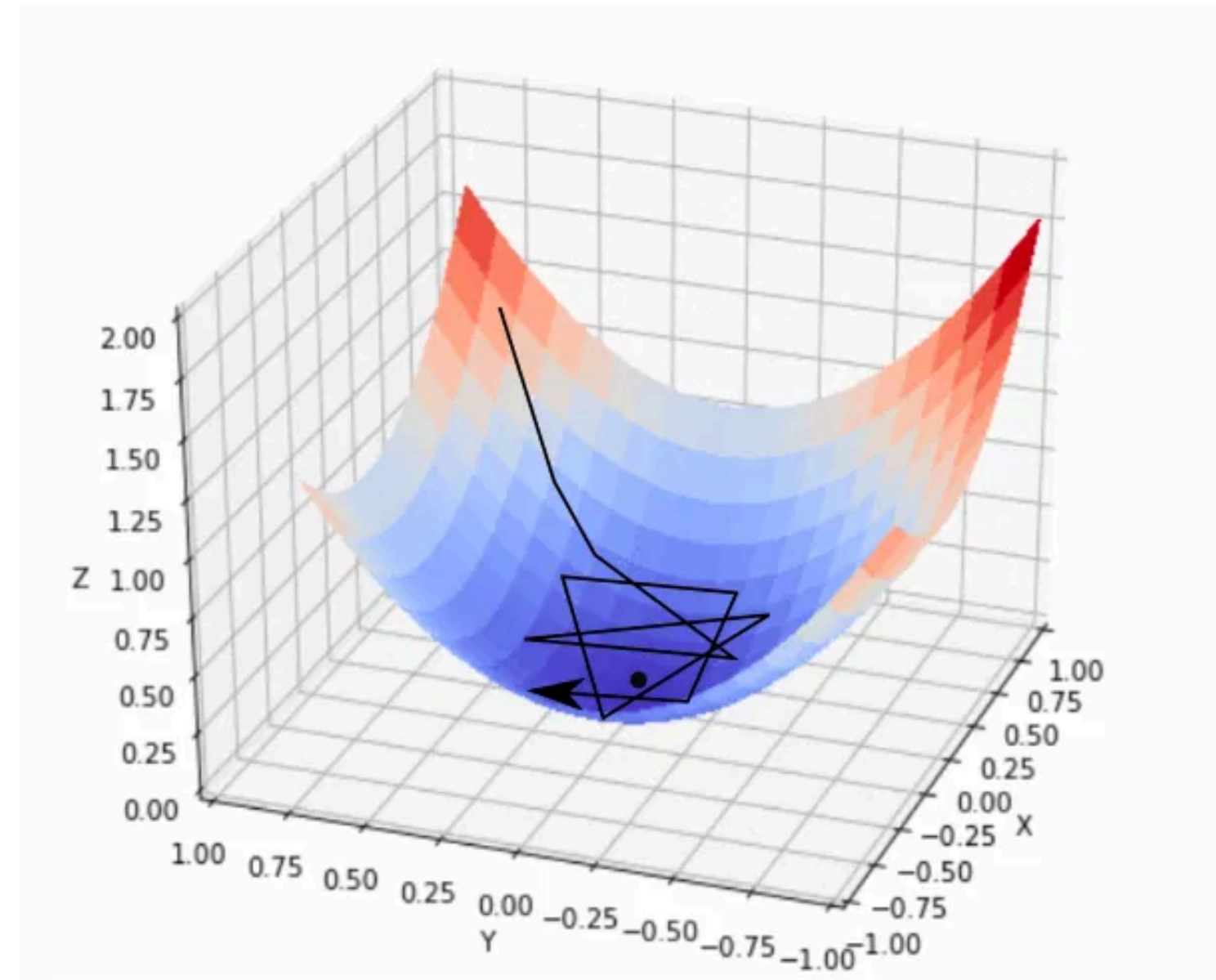
Descida do Gradiente: O Caminho para a Solução

O Conceito

É um algoritmo de otimização que ajusta os pesos na **direção oposta ao gradiente**. É como descer uma montanha (a função de custo) para encontrar o ponto mais baixo (o mínimo).

Taxa de Aprendizagem (Learning Rate - η)

Controla o tamanho do "passo" dado em cada ajuste. Um η muito grande pode fazer o algoritmo "saltar" o mínimo; um η muito pequeno torna o treinamento lento.



Descida do Gradiente: O Caminho para a Solução

Fórmula de Atualização

A regra que ajusta cada peso iterativamente:

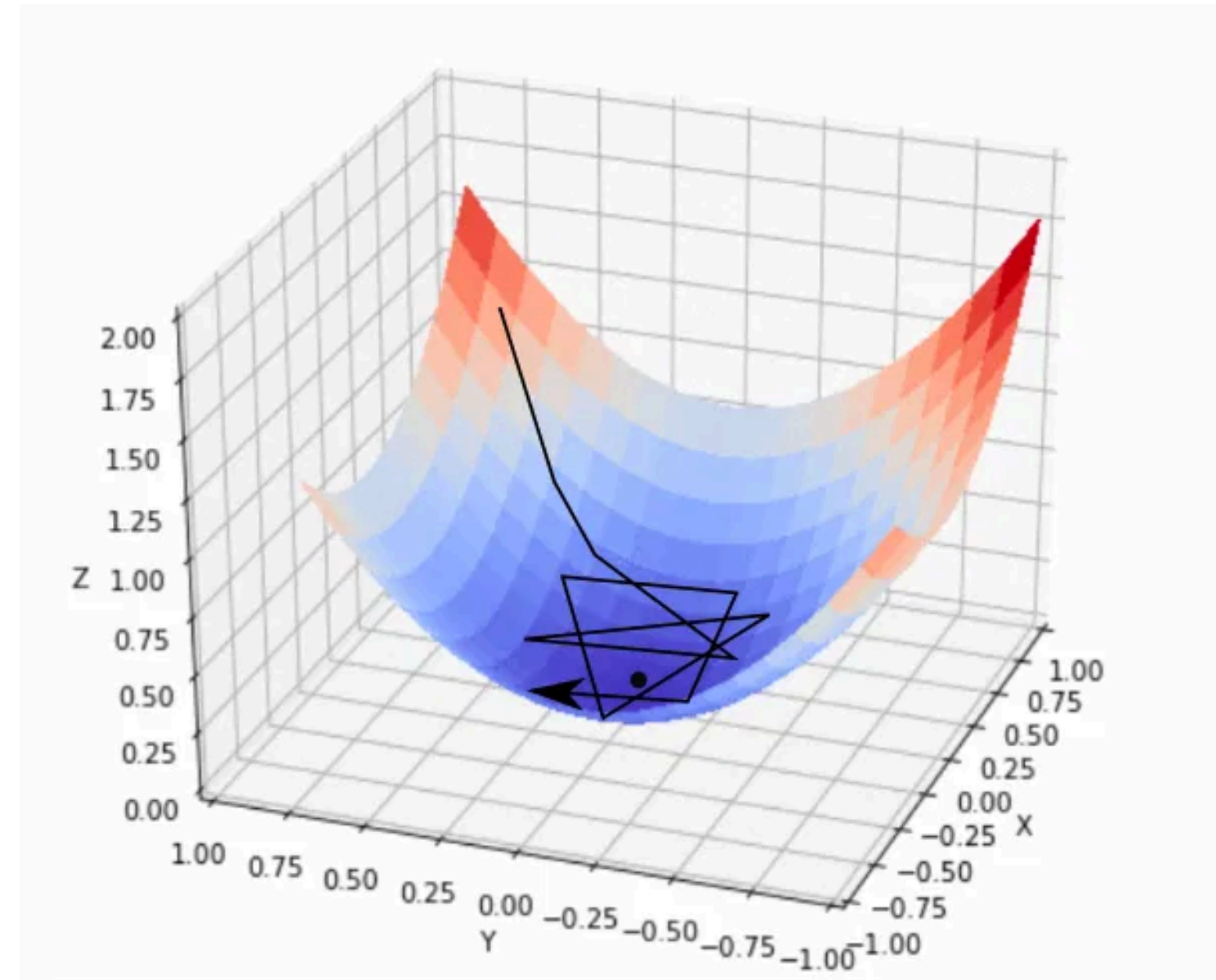
$$w_{\text{novo}} = w_{\text{antigo}} - \eta \cdot (\partial L / \partial w)$$

Interpretação

O gradiente ($\partial L / \partial w$) aponta na direção de maior aumento do erro. Subtraindo-o, movemos na direção de menor erro, aproximando-nos do mínimo.

O Desafio Prático:

Encontrar o η ideal é uma arte. Valores diferentes funcionam melhor para problemas diferentes. Algoritmos adaptativos (Adam, RMSprop) ajustam η automaticamente durante o treinamento.



Aplicações Práticas do MLP

CLASSIFICAÇÃO

Reconhecimento de Imagens

Classificação de dígitos escritos à mão (MNIST), reconhecimento de objetos em imagens digitais.

Detecção de Spam

Classificação de e-mails como spam ou legítimos baseado em características do texto.

Análise de Sentimentos

Classificação de textos em redes sociais como positivos, negativos ou neutros.

REGRESSÃO

Previsão de Preços

Estimativa de preços de imóveis, ações ou produtos baseado em características e histórico.

Séries Temporais

Previsão de valores futuros em dados temporais como temperatura, vendas ou tráfego.

Estimativa de Demanda

Previsão de demanda de produtos para otimização de estoque e planejamento.

RECONHECIMENTO DE PADRÕES

Detecção de Anomalias

Identificação de comportamentos anormais em sistemas, transações financeiras ou redes.

Diagnóstico Médico

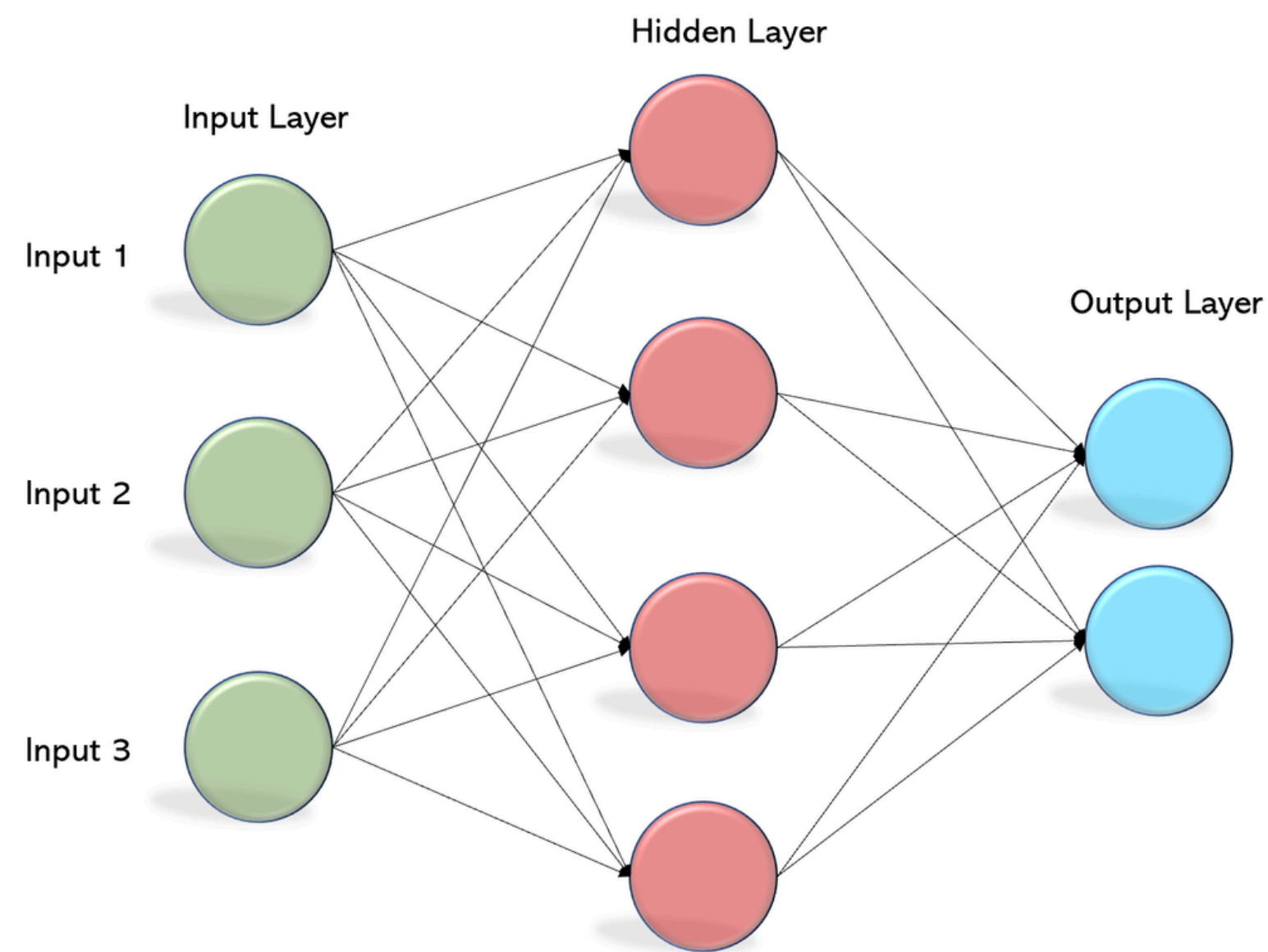
Análise de exames médicos para detecção de doenças e apoio ao diagnóstico.

Biometria

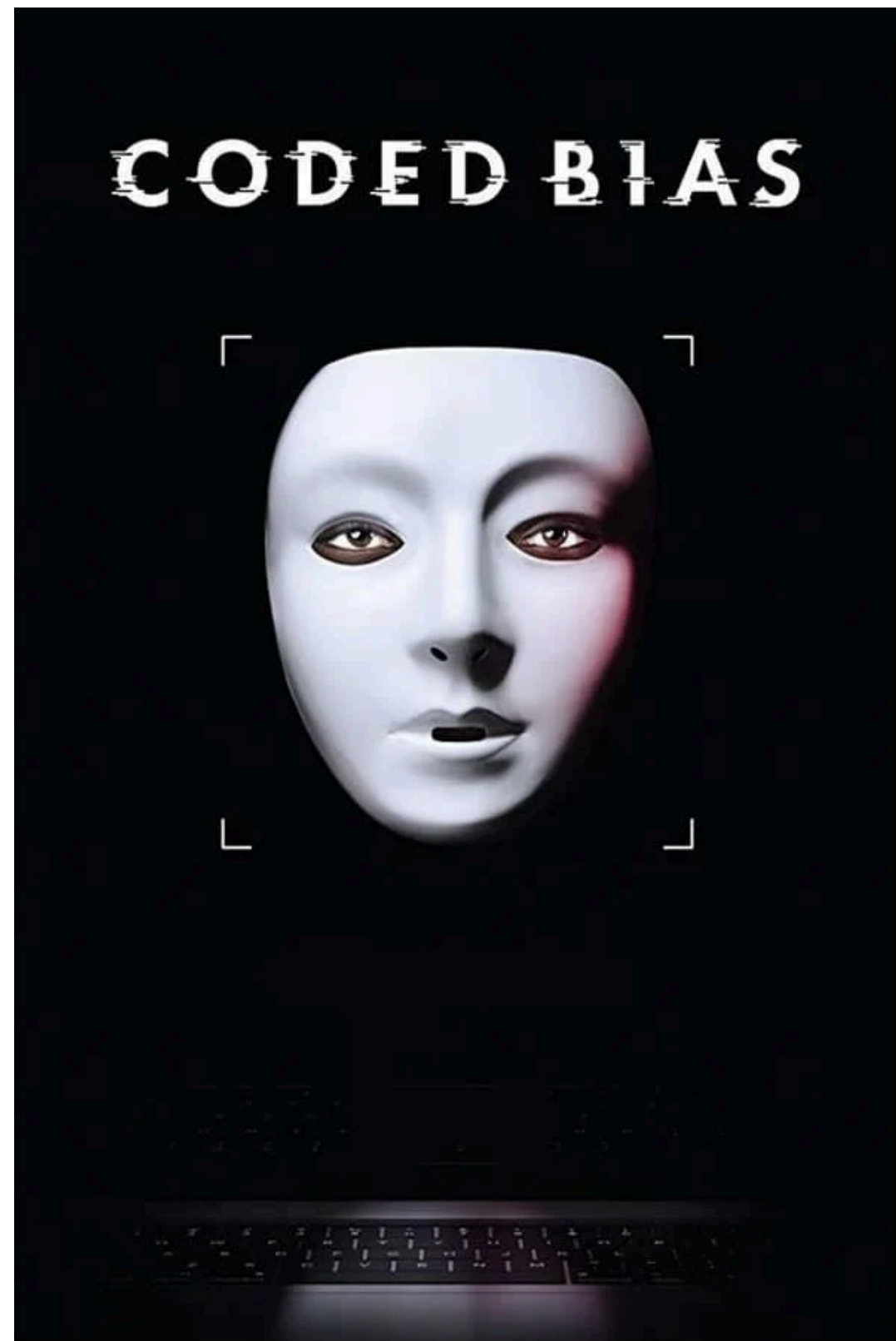
Reconhecimento facial, impressão digital e autenticação biométrica.

Por Que o MLP é Importante?

O MLP é a **arquitetura fundamental** que permitiu o avanço das Redes Neurais Artificiais. Embora frequentemente substituído por arquiteturas mais especializadas (CNNs para imagens, RNNs para sequências), o MLP permanece **essencial para a compreensão** de como redes neurais aprendem e é ainda amplamente utilizado em muitas aplicações práticas.



Coded bias (2020)



O filme mostra a jornada de Joy Buolamwini, pesquisadora do MIT Media Lab, que descobriu que os sistemas de reconhecimento facial frequentemente falham em identificar corretamente rostos de pessoas negras, especialmente mulheres.

Em uma das cenas iniciais do documentário, Buolamwini demonstra a falha de um sistema de análise facial ao posicionar seu rosto em frente a uma IA: quando coloca uma máscara branca, o sistema detecta a face; quando tira, nada acontece. "Coded Bias" mostra que as tecnologias não são neutras e que os algoritmos carregam os preconceitos existentes na sociedade, excluindo pessoas e reforçando estereótipos.

REFERÊNCIAS

1. FILHO, Oscar Gabriel. Inteligência Artificial e Aprendizagem de Máquina: aspectos teóricos e aplicações. 1. ed. São Paulo: Edgard Blücher, 2023. 462 p. ISBN 978-65-5506-620-3
2. HUYEN, Chip. Projetando sistemas de Machine Learning: processo iterativo para aplicações prontas para produção. Tradução de Cibelle Ravaglia. 1. ed. São Paulo: Alta Books, 2024. 384 p. ISBN 978-8550819679
3. MIENYE, I. D.; JERE, N. A survey of decision trees: concepts, algorithms, and applications. East London: Walter Sisulu University, 2017. Disponível em: <...>. Acesso em: 03 nov. 2025.

Dúvidas?



Profº Lindenberg Andrade
E-mail: linndemberg1@gmail.com



Additional contacts via QR code